

FORMATION EMBARQUÉE

TD 01 Langage C

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TD 01 — Langage C : énoncés et corrigés détaillés

Compile chaque corrigé toi-même (`gcc -Wall -Wextra`) et teste-le avec tes propres cas. Les corrigés sont volontairement commentés « comme en revue de code » : lis les commentaires autant que le code.

Exercice 1 — Inversion de l'ordre des bits

Énoncé. Écris `uint8_t inverse_bits(uint8_t x)` qui renverse l'ordre des bits : `0b10110000` → `0b00001101`.

Corrigé

```
#include <stdint.h>

uint8_t inverse_bits(uint8_t x) {
    uint8_t resultat = 0;
    for (uint8_t i = 0; i < 8; i++) {
        resultat <=< 1;           // faire de la place à droite
        resultat |= (x & 1);      // y copier le bit de poids faible de x
        x >>= 1;                 // passer au bit suivant de x
    }
    return resultat;
}
```

Comment ça marche : à chaque tour, le bit le plus à droite de `x` devient le bit le plus à droite de `resultat`, qui est ensuite poussé vers la gauche. Après 8 tours, le premier bit copié se retrouve tout à gauche → ordre inversé.

Test minimal (à prendre comme modèle pour tous tes exercices) :

```
#include <assert.h>

int main(void) {
    assert(inverse_bits(0xB0) == 0x0D); // l'exemple de l'énoncé
    assert(inverse_bits(0x00) == 0x00); // cas limites
    assert(inverse_bits(0xFF) == 0xFF);
    assert(inverse_bits(0x01) == 0x80);
    assert(inverse_bits(inverse_bits(0x5A)) == 0x5A); // involutif !
    return 0;
}
```

Variante « pro » (sans boucle, par échanges de blocs — utile en traitement de signal) :

```
uint8_t inverse_bits_rapide(uint8_t x) {
    x = (uint8_t)((x & 0xF0) >> 4 | (x & 0x0F) << 4); // échange les quartets
    x = (uint8_t)((x & 0xCC) >> 2 | (x & 0x33) << 2); // échange les paires
    x = (uint8_t)((x & 0xAA) >> 1 | (x & 0x55) << 1); // échange les bits
    return x;
}
```

Exercice 2 — Tampon circulaire (ring buffer)

Énoncé. Implémente un tampon circulaire de 32 octets avec `put()` et `get()` — la structure de données de tout driver UART.

Corrigé

```
#include <stdint.h>
#include <stdbool.h>

#define RB_TAILLE 32u // ASTUCE : une puissance de 2 permet le modulo par masque

typedef struct {
    uint8_t donnees[RB_TAILLE];
    volatile uint8_t tete; // index d'écriture (producteur, ex. ISR RX)
    volatile uint8_t queue; // index de lecture (consommateur, ex. main)
} RingBuffer;

// Convention : tete == queue → vide
//              (tete+1)%N == queue → plein (on "sacrifie" une case,
//              ce qui évite un compteur partagé)

void rb_init(RingBuffer *rb) {
    rb->tete = 0;
    rb->queue = 0;
}

bool rb_est_vide(const RingBuffer *rb) {
    return rb->tete == rb->queue;
}

bool rb_est_plein(const RingBuffer *rb) {
    return ((rb->tete + 1u) & (RB_TAILLE - 1u)) == rb->queue;
}

bool rb_put(RingBuffer *rb, uint8_t octet) {
    if (rb_est_plein(rb))
        return false; // on REFUSE plutôt qu'écraser
    rb->donnees[rb->tete] = octet;
    rb->tete = (rb->tete + 1u) & (RB_TAILLE - 1u); // modulo par masque
    return true;
}

bool rb_get(RingBuffer *rb, uint8_t *octet) {
    if (rb_est_vide(rb))
        return false;
    *octet = rb->donnees[rb->queue];
    rb->queue = (rb->queue + 1u) & (RB_TAILLE - 1u);
    return true;
}
```

Points de conception à retenir (c'est là qu'est la note !) :

1. **volatile** sur les index : dans l'usage réel, `put()` est appelé par une ISR et `get()` par la boucle principale — chacun doit voir les modifications de l'autre.
2. **Pourquoi c'est sûr sans section critique** (à un producteur / un consommateur) : `put()` n'écrit que `tete`, `get()` n'écrit que `queue`. Chacun ne fait que lire l'index de l'autre. Sur un index 8 bits, la lecture est atomique. À plusieurs producteurs : section critique obligatoire.

3. **Modulo par masque** `& (N-1)` : ne marche que si N est une puissance de 2, mais coûte 1 cycle au lieu d'une division.
 4. **Politique de saturation** : ici on refuse l'octet quand c'est plein (retour `false`). L'autre politique (écraser le plus ancien) se justifie pour des mesures dont seule la plus récente compte. **Choisir et documenter.**
-

Exercice 3 — FSM feu tricolore avec appel piéton

Énoncé. Feu : vert 5 s → orange 1 s → rouge 5 s, en boucle. Un « appui piéton » écourte le vert.

Corrigé

```
#include <stdint.h>
#include <stdbool.h>

typedef enum { VERT, ORANGE, ROUGE } EtatFeu;

typedef struct {
    EtatFeu  etat;
    uint32_t t_entree_etat; // horodatage d'entrée dans l'état courant
    bool     demande_pieton;
} Feu;

#define DUREE_VERT_MS      5000u
#define DUREE_ORANGE_MS   1000u
#define DUREE_ROUGE_MS    5000u
#define VERT_MINI_PIETON_MS 1000u // le vert dure au moins 1 s même sur appui

void feu_init(Feu *f, uint32_t maintenant) {
    f->etat = ROUGE;
    f->t_entree_etat = maintenant;
    f->demande_pieton = false;
}

// Appelée par l'ISR ou le scrutateur du bouton
void feu_appui_pieton(Feu *f) {
    f->demande_pieton = true; // on MÉMORISE : la FSM décidera quand agir
}

// À appeler très souvent (chaque tour de boucle). maintenant = millis().
void feu_tick(Feu *f, uint32_t maintenant) {
    uint32_t ecoule = maintenant - f->t_entree_etat; // robuste au débordement

    switch (f->etat) {
    case VERT:
        // Fin normale, OU appui piéton après le minimum de sécurité
        if (ecoule >= DUREE_VERT_MS ||
            (f->demande_pieton && ecoule >= VERT_MINI_PIETON_MS)) {
            f->etat = ORANGE;
            f->t_entree_etat = maintenant;
        }
        break;

    case ORANGE:
        if (ecoule >= DUREE_ORANGE_MS) {
            f->etat = ROUGE;
            f->t_entree_etat = maintenant;
            f->demande_pieton = false; // demande servie : le rouge arrive
        }
        break;

    case ROUGE:
        if (ecoule >= DUREE_ROUGE_MS) {
            f->etat = VERT;
        }
    }
```

```

        f->t_entree_etat = maintenant;
    }
    break;
}
}

```

Points de conception :

- L'appui piéton **ne change pas l'état lui-même** : il pose un drapeau que la FSM consomme. C'est la séparation « événements / transitions » — elle évite les états incohérents quand l'événement tombe au mauvais moment.
- `VERT_MINI_PIETON_MS` : dans un vrai système, on ne coupe jamais un feu instantanément (véhicule engagé). Penser aux **contraintes de sécurité** fait partie de la conception, pas de la finition.
- La demande est remise à zéro **quand elle est servie** (passage au rouge), pas quand elle est reçue.

Pour tester sur PC sans `millis()` : simuler le temps.

```

int main(void) {
    Feu f;
    feu_init(&f, 0);
    for (uint32_t t = 0; t < 30000; t += 100) { // 30 s simulées par pas de 100 ms
        if (t == 12000) feu_appui_pieton(&f); // appui à t=12 s
        feu_tick(&f, t);
        printf("%5u ms : etat=%d\n", t, f.etat);
    }
}

```

Exercice 4 — Décodage de trame

Énoncé. Décode la trame de 4 octets `{0xA5, id, valeur_hi, valeur_lo}` en structure, en vérifiant l'octet de tête.

Corrigé

```
#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>

#define TRAME_TETE      0xA5u
#define TRAME_LONGUEUR 4u

typedef struct {
    uint8_t id;
    uint16_t valeur;
} Mesure;

// Renvoie true si la trame est valide et remplit *sortie.
// Prend la longueur en paramètre : on ne fait JAMAIS confiance à l'appelant.
bool trame_decoder(const uint8_t *trame, size_t longueur, Mesure *sortie) {
    if (trame == NULL || sortie == NULL)        // 1. pointeurs valides
        return false;
    if (longueur != TRAME_LONGUEUR)            // 2. longueur exacte
        return false;
    if (trame[0] != TRAME_TETE)                 // 3. octet de synchronisation
        return false;

    sortie->id = trame[1];
    // Assemblage big-endian : octet de poids fort d'abord.
    // Le cast en uint16_t AVANT le décalage évite une promotion signée hasardeuse.
    sortie->valeur = (uint16_t)((uint16_t)trame[2] << 8) | trame[3];
    return true;
}
```

Les trois vérifications dans l'ordre (pointeurs → longueur → contenu) sont le squelette de tout parseur robuste. Un décodeur qui plante sur une trame corrompue est une faille : sur un bus réel, les trames corrompues *arrivent*.

Question bonus posée en entretien : pourquoi ne pas faire `Mesure *m = (Mesure*)trame;` ? Trois raisons : le padding/alignement de la structure ne correspond pas forcément aux 4 octets ; l'endianness de la machine peut différer de celle du protocole ; et déréférencer un pointeur mal aligné est un comportement indéfini sur certains processeurs (dont certains ARM). **On décode champ par champ.**

Exercice 5 (complément) — Chasse aux bugs

Énoncé. Chaque fragment contient au moins un bug. Trouve-le et corrige.

```

// A
uint8_t i;
for (i = 10; i >= 0; i--) { traiter(i); }

// B
char *message(void) {
    char buf[32];
    snprintf(buf, sizeof buf, "erreur %d", code);
    return buf;
}

// C
volatile uint16_t compteur_isr;           // incrémenté dans une ISR (CPU 8 bits)
uint16_t copie = compteur_isr;

// D
if (etat = ETAT_ERREUR) { alarme(); }

// E
#define CARRE(x) x * x
int y = CARRE(a + 1);

```

Corrigé

- **A** — `uint8_t` est **non signé** : `i >= 0` est toujours vrai (après 0, `i--` donne 255). Boucle infinie. Corrections possibles : `int8_t i`, ou boucle descendante idiomatique `for (uint8_t i = 11; i-- > 0; )`.
- **B** — retourne l'adresse d'un tableau **local** détruit à la sortie : comportement indéfini. Corriger en passant le buffer en paramètre (`void message(char *buf, size_t n)`) ou en le déclarant `static` (en documentant alors qu'il est écrasé à chaque appel et non réentrant).
- **C** — sur un CPU 8 bits, lire un 16 bits prend 2 instructions : l'ISR peut s'intercaler entre les deux (valeur « déchirée »). `volatile` n'empêche pas ça. Correction : section critique autour de la copie (désactiver/réactiver les interruptions).
- **D** — `=` au lieu de `==` : affecte puis teste la valeur (toujours vraie si `ETAT_ERREUR != 0`). Correction : `==`. Prévention : compiler avec `-Wall` (GCC le signale) ou écrire `ETAT_ERREUR == etat` (« conditions Yoda »).
- **E** — macro non parenthésée : `CARRE(a + 1)` devient `a + 1 * a + 1 = 2a + 1`. Correction : `#define CARRE(x) ((x) * (x))` — et encore, `x` y est évalué deux fois (`CARRE(i++)` reste faux) : préférer une fonction `static inline`.

Auto-évaluation avant le module 02

Sans notes, tu dois pouvoir : écrire set/clear/toggle/test d'un bit ; expliquer `volatile` et ses limites (pas d'atomicité) ; écrire un ring buffer complet ; dessiner puis coder une FSM ; lister les 3 vérifications d'un parseur ; expliquer pourquoi on bannit `malloc` après l'init.